

Post-Discharge App — Project Handoff

This document summarizes the product vision, architecture decisions, and implementation plan for a mobile app that helps patients navigate the 30 days after hospital discharge. Hand this to Claude Code to begin building.

1. Product Vision

Problem: Hospital discharge instructions are a 6-page PDF written at a 12th-grade reading level, handed to someone who just had surgery and is on pain meds. Most patients don't understand what to do, skip medications, miss follow-up signals, and end up readmitted — costing them their health and costing hospitals Medicare penalties.

Solution: A mobile app that:

- 1. Ingests discharge instructions (photo, PDF, or eventually Epic FHIR)
- 2. Translates them into plain-language daily tasks using AI
- 3. Runs condition-aware daily check-ins to catch red flags early
- 4. Sends medication reminders and logs adherence

Primary user: The patient themselves (not caregivers or providers — that's v2+).

Condition scope: General (any discharge type) to start. Specialize based on usage data later.

Go-to-market path: B2C first — patients find it themselves after going home scared. No hospital deal needed to launch. Use traction + readmission data to pitch hospitals for B2B2C partnerships in v2.

2. Core Features (v1 Scope)

Feature 1 — Instruction Translator (AI-powered)

The centerpiece of the app. Patient uploads or photographs their discharge paperwork. Claude API parses it and returns structured JSON:

```
json
```

```
{
  "medications": [
    {
      "name": "Metoprolol",
      "dose": "25mg",
      "frequency": "twice daily",
      "instructions": "Take with food",
      "times": ["08:00", "20:00"]
    }
  ],
  "activity_restrictions": [
    "No lifting over 10 lbs for 2 weeks",
    "No driving for 48 hours after procedure"
  ],
  "red_flags": [
    "Temperature above 101°F",
    "Chest pain or shortness of breath",
    "Wound drainage that is green or foul-smelling"
  ],
  "follow_up_appointments": [
    {
      "type": "Primary care",
      "timeframe": "Within 7 days"
    }
  ],
  "diet_restrictions": [],
  "wound_care": []
}
```

This structured output drives everything else in the app — check-in questions, reminders, and task lists all pull from it.

Feature 2 — Daily Check-In

A short push-notification-triggered symptom check each morning. Questions are dynamically generated from the patient's specific red flags and restrictions — not a generic wellness survey.

Example: if discharge instructions flagged fever risk → morning check-in asks "Have you had a temperature above 101°F?" If yes → "You should contact your care team today" nudge, with a tap-to-call option for their provider's number.

Feature 3 — Medication Reminders + Log

- Push notifications at scheduled medication times (pulled from translated instructions)

- One-tap "taken" checkoff
- Log view showing adherence history
- Missed dose nudge after 30-minute window

Out of scope for v1

- Caregiver/family visibility dashboard
 - Epic FHIR auto-pull (requires hospital partnership + App Orchard)
 - Care team monitoring portal
 - In-app chat with providers
 - Wearable integrations
-

3. Technical Architecture

Platform

React Native (or Flutter) — both iOS and Android from a single codebase. Patients are at home on mobile; no desktop version needed for v1.

Input Methods

Method	v1	Notes
Photo / camera scan	✓	Use React Native camera + send image to Claude API
PDF upload	✓	File picker, convert to base64, send to Claude API
Epic FHIR auto-pull	✗ v2	Requires Epic App Orchard listing + hospital sponsor

AI Layer

Claude API (`claude-sonnet-4-20250514`) for the instruction translator.

Recommended prompt structure:

You are a medical document parser. Extract structured information from the following hospital discharge instructions. Return ONLY valid JSON with no preamble, matching this schema: [schema]. If a field has no information, return an empty array. Simplify all instructions to a 6th-grade reading level.

Document:

[base64 image or extracted PDF text]

For photo inputs: send as `image` content block (base64 JPEG/PNG). For PDFs: extract text first (see PDF handling below), then send as text.

Backend

HIPAA compliance is non-negotiable from day one. Even in beta, if the app stores any health information, you need a signed Business Associate Agreement (BAA) with your cloud provider.

Recommended stack:

- **Cloud:** AWS with a signed BAA (most common path for indie health apps)
 - Alternative: Google Cloud Healthcare API, or Aptible (managed HIPAA platform, higher cost but easier)
- **Database:** Amazon RDS (PostgreSQL) with encryption at rest + in transit
- **Auth:** AWS Cognito or Auth0 — both support MFA, which is expected for health apps
- **Push notifications:** AWS SNS or Expo Notifications (if using Expo/React Native)
- **File storage:** S3 with server-side encryption (SSE-S3 or SSE-KMS), versioning enabled

Do not use Firebase — Google's standard Firebase does not cover HIPAA by default and requires special configuration + a BAA that is not always available on free tiers.

Data Model (simplified)

```
users
  id, email, created_at, timezone

discharges
  id, user_id, raw_input_type (photo|pdf|fhir), raw_input_url,
  parsed_json, created_at, discharge_date, provider_phone

medications
  id, discharge_id, name, dose, frequency, times[], instructions

check_ins
```

```
id, user_id, discharge_id, date, responses_json, red_flag_triggered,
completed_at
```

```
medication_logs
```

```
id, medication_id, user_id, scheduled_time, taken_at, skipped
```

Security requirements

- All API calls over HTTPS only
 - Health data encrypted at rest (AES-256)
 - JWT tokens with short expiry + refresh token rotation
 - No health data in analytics events (Mixpanel, Amplitude etc.) — log behavior only, never content
 - App Store / Play Store privacy labels must accurately reflect PHI collection
-

4. Project Structure (React Native)

```
/src
  /api
    claude.ts      # Instruction translator API call
    auth.ts        # Login, register, token refresh
    discharge.ts   # Save/fetch discharge records
    checkin.ts     # Submit daily check-in
  /screens
    Onboarding/    # Welcome, permissions, account creation
    Upload/        # Camera scan + PDF upload flow
    Processing/    # "Reading your instructions..." loading state
    Home/          # Today's tasks + medication schedule
    CheckIn/       # Daily symptom check-in flow
    MedLog/        # Medication history
    Instructions/  # Full plain-language instruction view
    Settings/      # Profile, notifications, emergency contact
  /components
    TaskCard.tsx
    MedReminderCard.tsx
    RedFlagAlert.tsx
    CheckInQuestion.tsx
  /hooks
    useDischarge.ts
    useCheckIn.ts
```

```
    useMedications.ts
    useNotifications.ts
  /utils
    pdfExtract.ts      # PDF → text extraction
    imageToBase64.ts
    parseClaudeResponse.ts
  /navigation
    AppNavigator.tsx
  /store
    auth.ts            # Zustand or Redux slice
    discharge.ts
```

5. Claude API — Instruction Translator Implementation

```
typescript
```

```

// src/api/claude.ts

const CLAUDE_API_URL = 'https://api.anthropic.com/v1/messages';

export async function parseDischargeInstructions(
  input: { type: 'image'; base64: string; mediaType: string } | { type: 'text'; content: string }
): Promise<DischargeJSON> {
  const content =
    input.type === 'image'
      ? [
          {
            type: 'image',
            source: { type: 'base64', media_type: input.mediaType, data: input.base64 },
          },
          { type: 'text', text: 'These are hospital discharge instructions. Extract structured information from the text below. Return ONLY valid JSON – no preamble, no markdown fences – matching this exact schema: ' + JSON.stringify({
            "medications": [{ "name": string, "dose": string, "frequency": string, "instructions": string[] },
            "activity_restrictions": string[],
            "red_flags": string[],
            "follow_up_appointments": [{ "type": string, "timeframe": string }],
            "diet_restrictions": string[],
            "wound_care": string[]
          })
        ]
      : [{ type: 'text', text: input.content }];

  const response = await fetch(CLAUDE_API_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'x-api-key': process.env.ANTHROPIC_API_KEY!,
      'anthropic-version': '2023-06-01',
    },
    body: JSON.stringify({
      model: 'claude-sonnet-4-20250514',
      max_tokens: 2000,
      system: `You are a medical document parser. Extract structured information from the text below. Return ONLY valid JSON – no preamble, no markdown fences – matching this exact schema: ' + JSON.stringify({
        "medications": [{ "name": string, "dose": string, "frequency": string, "instructions": string[] },
        "activity_restrictions": string[],
        "red_flags": string[],
        "follow_up_appointments": [{ "type": string, "timeframe": string }],
        "diet_restrictions": string[],
        "wound_care": string[]
      })
    })
  });

  const json = await response.json();
  return parseDischargeJSON(json.content[0].text);
}

function parseDischargeJSON(json: string): DischargeJSON {
  try {
    return JSON.parse(json);
  } catch {
    return {
      medications: [],
      activity_restrictions: [],
      red_flags: [],
      follow_up_appointments: [],
      diet_restrictions: [],
      wound_care: []
    };
  }
}

Rewrite all instructions at a 6th-grade reading level. If a field has no data, return null.

messages: [{ role: 'user', content }],
  }),
});

```

```
const data = await response.json();
const text = data.content.find((b: any) => b.type === 'text')?.text ?? '{}';
return JSON.parse(text) as DischargeJSON;
}
```

Important: The API key must never be in the mobile app binary. Route Claude API calls through your own backend — the mobile app calls your server, your server calls Anthropic. This also lets you log errors and monitor costs.

6. HIPAA Checklist (Before Launch)

- ☐ Sign BAA with cloud provider (AWS / GCP / Aptible)
 - ☐ Encryption at rest enabled on all databases and file storage
 - ☐ All data in transit uses TLS 1.2+
 - ☐ Auth tokens short-lived with refresh rotation
 - ☐ Audit logging enabled (who accessed what, when)
 - ☐ Privacy policy written and reviewed (must specifically mention PHI)
 - ☐ Terms of service written
 - ☐ App Store privacy nutrition label filled out accurately
 - ☐ No PHI in crash logs or analytics
 - ☐ Delete-my-account flow implemented (HIPAA + App Store requirement)
 - ☐ Penetration test or security review before scaling beyond beta
-

7. Implementation Sequence

Phase 1 — Core loop (weeks 1-4)

1. Set up React Native project with TypeScript
2. HIPAA-compliant backend (AWS + BAA, RDS, Cognito)
3. Photo capture + PDF upload screens
4. Claude API integration (instruction translator) — routed through backend
5. Parsed instruction display screen (plain-language task list)

Phase 2 — Engagement loop (weeks 5-8)

6. Medication reminder system (local push notifications first, then AWS SNS)
7. Medication log + checkoff UI

- 8. Daily check-in flow (dynamic questions from red_flags array)
- 9. Red flag alert screen + tap-to-call provider

Phase 3 — Polish + launch (weeks 9-12)

- 10. Onboarding flow
- 11. Settings (notifications, emergency contact, account)
- 12. TestFlight / Play Store internal test
- 13. App Store submission

Phase 4 — v2 (post-launch)

- 14. Epic FHIR integration (requires App Orchard application)
- 15. Caregiver sharing / family visibility
- 16. Care team dashboard (pivot to B2B2C)
- 17. Condition-specific check-in templates (cardiac, ortho, etc.)

8. Key Decisions & Rationale

Decision	Choice	Reason
Platform	React Native / Flutter	Single codebase, both platforms, strong ecosystem
AI model	Claude API (Sonnet)	Best-in-class document parsing, vision capability for photos
API routing	Backend proxy	Never expose API keys in mobile binary
Cloud	AWS + BAA	Industry standard for HIPAA, most documentation/community
Auth	Cognito or Auth0	MFA support, HIPAA-aligned, managed service
Database	RDS PostgreSQL	Relational model fits discharge → meds → logs hierarchy
Go-to-market	B2C first	No hospital deal needed; validate product before complex sales
FHIR	v2 only	Requires Epic App Orchard + hospital sponsor; too slow for v1

Decision	Choice	Reason
Condition	General	Widest learning surface; specialize from data scope

9. Risks & Mitigations

"This is not medical advice" liability — Every screen that surfaces health information must include a clear disclaimer. The app helps patients follow instructions they already received from a doctor; it does not diagnose or recommend treatment.

OCR / parsing errors — Claude can misread handwritten or poorly scanned documents. Build a "review your instructions" screen where patients can see the extracted tasks before they're saved, and tap to flag or correct anything.

Notification fatigue — Don't over-notify. One morning check-in, med reminders only at scheduled times, no marketing push in v1.

Elderly users — A meaningful share of discharged patients are elderly. Design for large tap targets (minimum 44×44pt), high contrast, and font size that respects system accessibility settings.

App Store review — Health apps get extra scrutiny. Have your HIPAA docs, privacy policy, and medical disclaimer ready before submission. Do not use the word "diagnose" anywhere.

10. Reference Links

- Anthropic API docs: <https://docs.anthropic.com>
- Epic App Orchard (for future FHIR): <https://apporchard.epic.com>
- AWS HIPAA eligible services: <https://aws.amazon.com/compliance/hipaa-eligible-services-reference/>
- Aptible (managed HIPAA platform): <https://www.aptible.com>
- React Native docs: <https://reactnative.dev>
- CMS readmission penalty program (why hospitals care): <https://www.cms.gov/medicare/payment/prospective-payment-systems/acute-inpatient-pps/hospital-readmissions-reduction-program-hrrp>